COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	7/17/26	**DATE SUBMITTED:**	7/22/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Table 2-1 Initial Driver Program

Screenshot

```
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        Student(std::string newName = "John Doe",int
newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called.\n";
        };
        ~Student(){
            std::cout << "Destructor Called.\n";
        };
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called\n";
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        Student& operator=(const Student &copyStudent){
            std::cout << "Assignment operator
called\n";
            if (this != &copyStudent) {
```

```
                studentName = copyStudent.studentName;
                studentAge = copyStudent.studentAge;
            };

            return *this;
        }

        void printDetails(){
            std::cout << this->studentName << " " <<
this->studentAge << std::endl;
        }
};


int main(){
    Student student1("Roman",28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Assignment operator called
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation	I observed from the code of the initial driver program that it tries to call all of the constructor, copy constructor, assignment operator, and destructor methods. The first variable, student1, creates a new object on its own, which is why it calls the constructor method. The second object creates a new object while using student1 as a parameter. This automatically calls the copy constructor. The third object is created and assigned

	the value of student2. Since student3 is already made and we are just assigning/copying the value of student2 into it, the copy assignment operator is called. Lastly, when it reaches the end of the program, the objects are destroyed.

Table 2-2. Modified Driver Program with Student Lists

Screenshot

```
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        Student(std::string newName = "John Doe",int
newAge=18){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor Called.\n";
        };
        ~Student(){
            std::cout << "Destructor Called.\n";
        };
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called\n";
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
        Student& operator=(const Student &copyStudent){
            std::cout << "Assignment operator
called\n";
            if (this != &copyStudent) {
                studentName = copyStudent.studentName;
                studentAge = copyStudent.studentAge;
            };

            return *this;
        }
```

```
        void printDetails(){
            std::cout << this->studentName << " " <<
this->studentAge << std::endl;
        }
};

int main(){
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy",
"Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

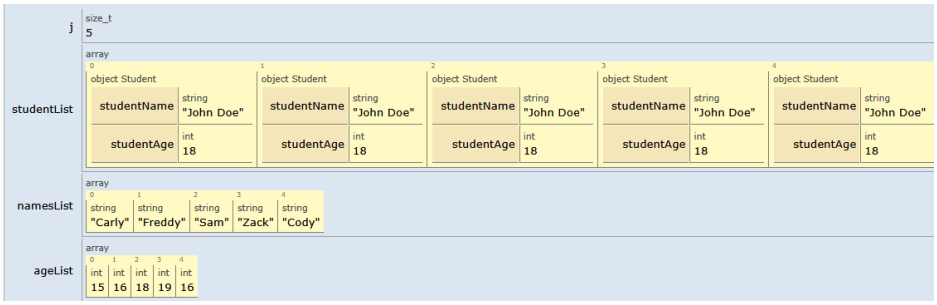
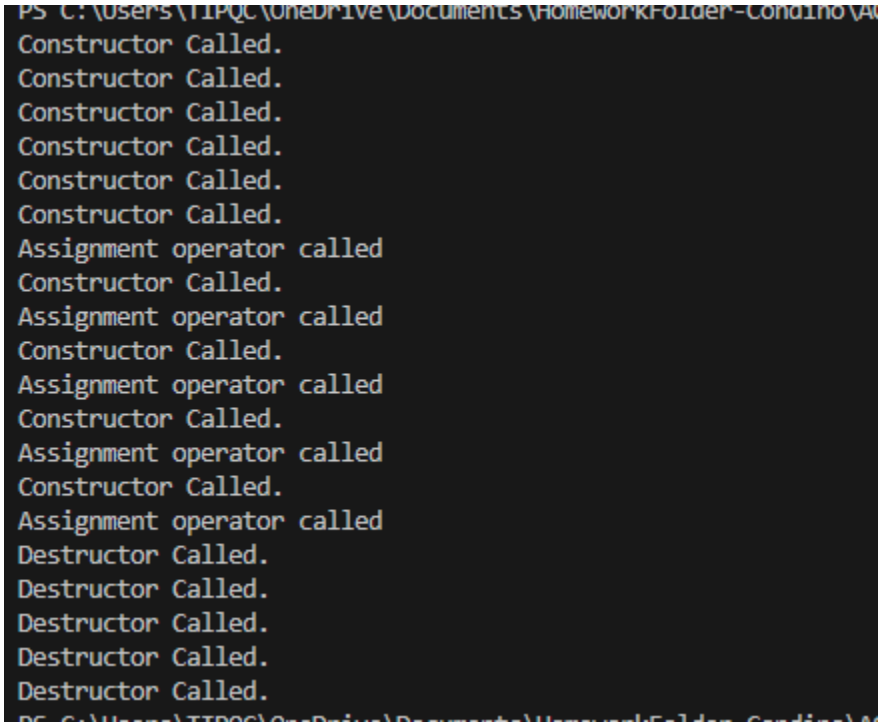
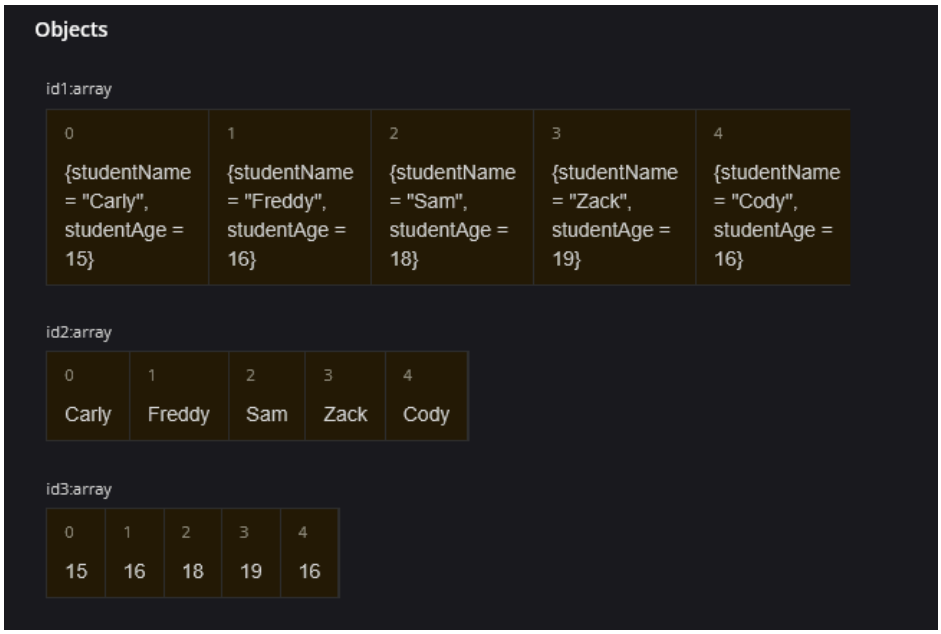
	
Observation	The class is the same as the initial driver's code. Now in the main function, the variable for size is first created with a data type of size_t. Next, the studentList array is created. The program automatically creates objects inside this array, meaning the constructor will be called. Also, since we have put default arguments for the name and age, when we have not put our own argument, the class defaults to those values. Hence, in the third picture, the array is populated with objects named "John Doe" with age 18. The arrays for names and age were then created. Lastly, at the end of the program, the objects were destroyed, so the destructor was called.

Table 2-3. Final Driver Program

Loop A	```
	int main(){
	    const size_t j = 5;
	    Student studentList[j] = {};
	    std::string namesList[j] = {"Carly", "Freddy",
	"Sam", "Zack", "Cody"};
	    int ageList[j] = {15, 16, 18, 19, 16};
	//Loop 1
	    for (size_t i = 0; i < j; i++){
	        Student *ptr = new
	Student(namesList[i],ageList[i]);
	        studentList[i] = *ptr;
	        delete ptr;
	        ptr = nullptr;
	    }
	
	    return 0;
	}
	``

<table>
<tr>
<td></td>
<td>




</td>
</tr>
<tr>
<td>Observation</td>
<td>After creating the default array, which is populated by “John Doe”, the loop creates a dynamic object from the class of Student, then gets the name and age from the same index as it is on the name and age list. This new object’s value is copied and placed in the array using the copy assignment operator on the line studentList[i] = *ptr. Hence, in the output, it is shown that the copy assignment operator is called. The third image shows the state of the array after the first loop.</td>
</tr>
</table>

Loop B	```int main(){
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};
//Loop 1
   for (size_t i = 0; i < j; i++){
        Student *ptr = new Student(namesList[i],ageList[i]);
        studentList[i] = *ptr;
        delete ptr;
        ptr = nullptr;
    }
//Loop 2
    for (size_t a = 0; a < j; a++){
        studentList[a].printDetails();
    }

    return 0;
}
```
```Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
``` |
| Observation | The second loop just loops through the student list, then prints their details using the printDetails() method. It traverses the array, then calls printDetails() to output the current values of the objects in the array. |

Modifications	Modification 1: ```Student& operator=(const Student &copyStudent){
std::cout << "Assignment operator called\n";
if (this != &copyStudent) {
``
```

```
                studentName = copyStudent.studentName;
                studentAge = copyStudent.studentAge;
            };

            return *this;
        }
```

Modification 2:

```
    for (size_t i = 0; i < j; i++ ){
        Student *ptr = new
Student(namesList[i],ageList[i]);
        studentList[i] = *ptr;
        delete ptr;
        ptr = nullptr;
    }

    for (size_t a = 0; a < j; a++){
        studentList[a].printDetails();
    }
```

Observation	For the first modification: I added a copy assignment constructor in the Student class. I made this so that a more predictable method will be called when copying a value by assigning it from another object. For the second modification: I underlined all the modifications. First, the index value's data type is the same as the data type of the "size," which is size_t. This is in order to have predictable behaviour because a comparison is done on the same data type. The second modification I made is to delete the new objects and set the pointer to null. This is made to save memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <map>
#include <string>
#include <span>

//Base Class
class Consumable{
   protected:
        std::string name;
        int addedQuantity;
        float price;
   public:
        virtual ~Consumable() = default;
        virtual float getItemPrice() = 0;
        virtual void printDetails() = 0;
        virtual std::string getName() = 0;
        virtual int getQuantity() const = 0;
};

//Implement templating for individual quantity map
template <typename Foods>
class Food: public Consumable {
       public:
            static std::map<std::string, int> quantity;
            float getItemPrice() override;
            void  printDetails() override;
            std::string getName() override;
            int getQuantity() const override;
};

//FRUIT AND VEGETABLE CLASSES

class Fruit: public Food<Fruit> {
   public:
        Fruit(std::string foodName="Unknown",float foodPrice=0,int
foodQuantity=1);
       ~Fruit() override;
       Fruit(const Fruit &copyFruit);
```

```
        Fruit& operator=(const Fruit &copyFruit);
};

class Vegetable: public Food<Vegetable>{
   public:
        Vegetable(std::string Foodname="Unknown",float Food_price=0,int
Food_quantity=1);
        ~Vegetable() override;
        Vegetable(const Vegetable &copyVegetable);
        Vegetable& operator=(const Vegetable &copyVegetable);
};

//Function prototype

int getTotalSum(std::span<Consumable*> GroceryList);
```

Figure 1: Headers and Class Declaration

This part shows the libraries and classes I used throughout the code. The map library was used to store the quantities of the objects, and the span library was used to pass the grocery list array without explicitly assigning it a size. The Classes are all derived from the base class Consumable. This class has the protected attributes name, addedQuantity, and price, which will all be useful later on when constructing the class. The methods were initialized with the virtual keyword so that they can be overridden when polymorphism happens. A template class was created so that I can create a public “quantity” map that is meant to be shared only with their own classes. This is declared with the keyword static so the fruit and vegetable classes have their own “quantity” map without them sharing. Meaning the fruit class have their own quantity map and vegetables have their own also. The methods were also defined in this class. The fruit and vegetable classes have their constructor, destructor, copy constructor, and copy assignment operator declared in this section. The function that will be used later to get the total sum of items inside the array was also declared here.

```
//================================== DEFINITION
====================================

//Declare outside class
template <typename Foods>
std::map<std::string, int> Food<Foods>::quantity;

template <typename Foods>
float Food<Foods>::getItemPrice(){
   return price*quantity[name];
```

```
}
template <typename Foods>
void Food<Foods>::printDetails(){
   std::cout << "Name: "<< name << " Price: " << price << " Quantity: "
<< quantity[name] <<"\n";
}
template <typename Foods>
std::string Food<Foods>::getName(){
   return name;
}
template <typename Foods>
int Food<Foods>::getQuantity() const{
   return quantity[name];
}

//============================================== Fruit METHODS
==============================================

Fruit::Fruit(std::string foodName,float foodPrice,int foodQuantity){
   name = foodName;
   price = foodPrice;
   addedQuantity = foodQuantity;
   quantity[foodName] += foodQuantity;
}

Fruit::~Fruit(){
   quantity[name]-= addedQuantity;
}

Fruit::Fruit(const Fruit &copyFruit){
   name = copyFruit.name;
   price = copyFruit.price;
   addedQuantity = copyFruit.addedQuantity;
}

Fruit& Fruit::operator=(const Fruit &copyFruit) {
   if (this == &copyFruit) {
       return *this;
```

```
    }

    name = copyFruit.name;
    price = copyFruit.price;
    addedQuantity = copyFruit.addedQuantity;

    return *this;
}

//============================================== VEGETABLE METHODS
=======================================

Vegetable::Vegetable(std::string foodName,float foodPrice,int
foodQuantity){
    name = foodName;
    price = foodPrice;
    addedQuantity = foodQuantity;
    quantity[name]+= foodQuantity;
}

Vegetable::~Vegetable(){
    quantity[name] -= addedQuantity;
}

Vegetable::Vegetable(const Vegetable &copyVegetable){
    name = copyVegetable.name;
    price = copyVegetable.price;
    addedQuantity = copyVegetable.addedQuantity;
}

Vegetable& Vegetable::operator=(const Vegetable &copyVegetable){
    if (this == &copyVegetable) {
        return *this;
    }

    name = copyVegetable.name;
    price = copyVegetable.price;
    addedQuantity = copyVegetable.addedQuantity;
```

```
    return *this;
}

//Problem 3 Solution
int getTotalSum(std::span<Consumable*> GroceryList){
    int totalSum = 0;

    for (const Consumable* item: GroceryList){
        if (item == nullptr) continue;
        totalSum ++;
    }

    return totalSum;
}
```

Figure 2: Method and function definition

This section shows the definition of the functions declared at the top of the file. The static map that was declared on the template class Food was defined here. The different methods for accessing the protected attributes, printing the details, and getting the total price for the item were defined here. The methods for the classes of fruits and vegetables were defined here. Their constructor, destructor, copy constructor, and copy assignment operator were all the same in function. The constructor sets the attributes, then adds their quantity to the quantity map. The destructor removes the quantity added to the map for that specific object. Both the copy constructor and copy assignment just copy the attributes from a given original value object to the copied object. The last function just takes in an array as an argument, traverses it, counts the objects if it is not a null pointer, then returns the count.

```
//================================== MAIN
==================================

int main() {
    const int arr_size = 4;
    //Problem number 1
    Fruit *apple = new Fruit("Apple",10,7);
    Fruit *banana = new Fruit("Banana",10,8);
    Vegetable *broccolli = new Vegetable("Broccolli",60,12);
    Vegetable *lettuce = new Vegetable("Lettucce",50,10);

    std::cout << "\nOriginal List\n";

    //Problem number 2
```

```
    Consumable* GroceryList[arr_size]  =
{apple,banana,broccolli,lettuce};

    for (const auto& item:GroceryList){
        item->printDetails();
    }

    //Problem number 3
    int totalSumObj = getTotalSum(GroceryList);

    std::cout << "Number of objects in the array: " << totalSumObj <<
"\n";

    //Problem number 4
    delete lettuce;
    lettuce = nullptr;
    GroceryList[3] = nullptr;

    std::cout << "\nNew List\n";

    for (const auto& item:GroceryList){
        if (item == nullptr) continue;
        item->printDetails();
    }

    int totalSumObj2 = getTotalSum(GroceryList);//Check the total objects
now

    std::cout << "Number of objects in the array: " << totalSumObj2 <<
"\n";

    //Cleanup
    for (auto& item:GroceryList){
        if (item == nullptr) continue;
        delete item;
        item = nullptr;
    }
```

```
    apple = banana = nullptr;
    broccolli = nullptr;


    return 0;
}
```

Figure 3: Driver Code

This driver code was created to implement the solutions to the given problem. It first initializes the array size to 4, then creates the objects apple, banana, broccoli, and lettuce on the heap. It then stores these objects in the array named GroceryList, which has type consumables so that it can accommodate the derived classes. It then calls the printDetails() method from these objects. Then the totalSum function was called to check the number of objects in the GroceryList array. It then deletes the lettuce object, sets the pointer for it in the variable, and the pointer in the array to null to avoid it being a dangling pointer. It then prints out the objects in the array again to make sure the code worked. Lastly, the code does a cleanup by deleting all the objects created in the heap and sets the pointers to null.

```
~/Downloads/HOA/HW # 2.1
❯ cd "/home/nielc/Downloads/HOA/HW # 2.1/" && g++ -std=c++20 fruits&vegetable.cpp -o fruits&vegetable && "/home/nielc/Downloads/HOA/HW # 2.1/"fruits&vegetable
Original List
Name: Apple Price: 10 Quantity: 7
Name: Banana Price: 10 Quantity: 8
Name: Broccolli Price: 60 Quantity: 12
Name: Lettucce Price: 50 Quantity: 10
Number of objects in the array: 4

New List
Name: Apple Price: 10 Quantity: 7
Name: Banana Price: 10 Quantity: 8
Name: Broccolli Price: 60 Quantity: 12
Number of objects in the array: 3
```

Figure 4: Driver Code Output

The figure above shows the output of the code from Figure 3. It shows the objects in the original list along with their attributes. After that, it also shows the output after removing the lettuce from the list and deleting it from the heap. This shows the expected behaviour from the code and confirms the code is working as expected

Conclusion

In this activity, I was able to explore more about C++. I was able to learn about dynamic memory allocation and the memory layout. I was able to used the keyword “new” to allocate objects on the heap while also deleting these objects when the object is done to prevent memory leaks. This can help with allocating memory at runtime that cannot be determined at compile timeile time. I was also able to implement my learnings on OOP in this program, such as the inheritance and polymorphism concepts of OOP. I was also able to learn about templating a class and learn about how it works. Templates are not a normal class, but are just blueprints of this class. I also learned how to create constructors, destructors, copy constructor, and a copy assignment operator by learning how they are named as methods and when they are invoked. Overall, I was able to learn more about how to implement object-oriented programming in C++, and also how to implement dynamic memory allocation to create objects at runtime instead, while maintaining the program's memory management by deleting objects not in use

.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.